

Foundations of Distributed Systems

Distributed Snapshot, Byzantine Agreement, Group Communication

Chapter 3.2 / 2025-10-02

Prof. Dr. Christian Tschudin

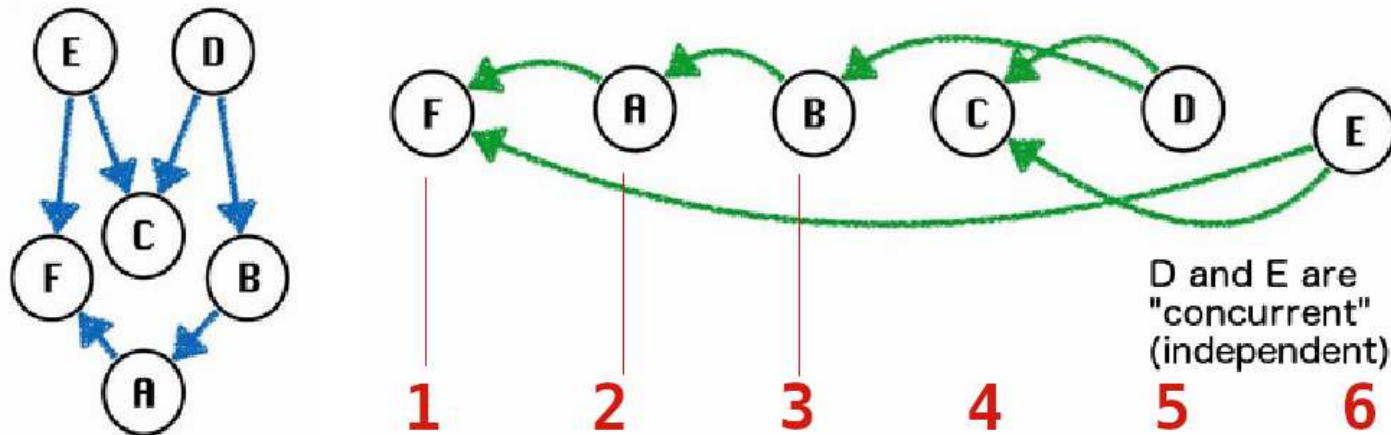
Dept of Mathematics and Computer Science, University of Basel

Overview

- **Recap: Distributed linear time?**
- Distributed snapshot
- Byzantine agreement
 - two general's problem
 - byzantine failures
 - byzantine problem and its impossibility result
- Group communication:
 - Reliable Broadcast

Recap: Distributed Linear Time?

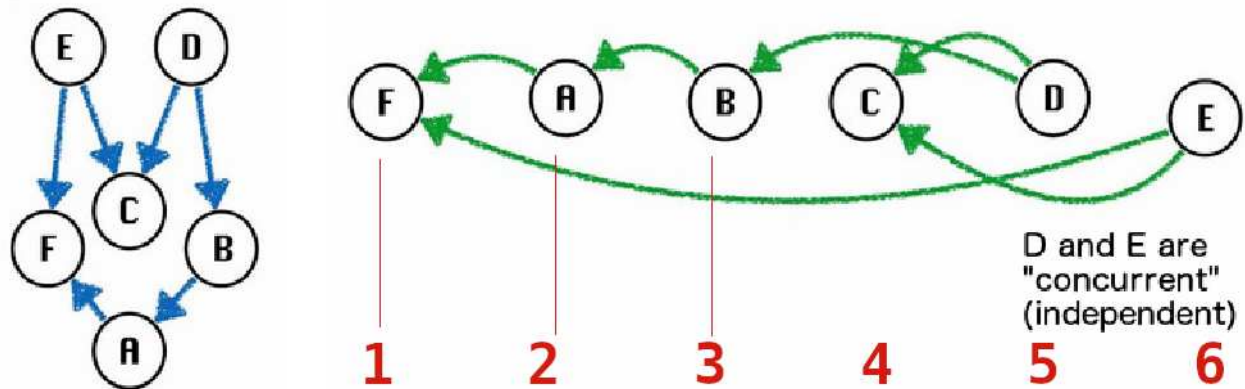
From last week: “Tangle”, DAG (causality graph) → use *topological sort* to serialize a DAG



Does this mean that we *can* get back a notion of linear time?

Recap: Distributed Linear Time?

From last week: “Tangle”, DAG (causality graph) → use *topological sort* to serialize a DAG



Does this mean that we *can* get back a notion of linear time?

- No, it's a “different sort of time”
- Must (arbitrarily) sort concurrent events and learning about new events *can change linearization!*

Global State

No global time available (can't reconstruct *exact* physical time over asynchronous channels) → no accurate global state obtainable.

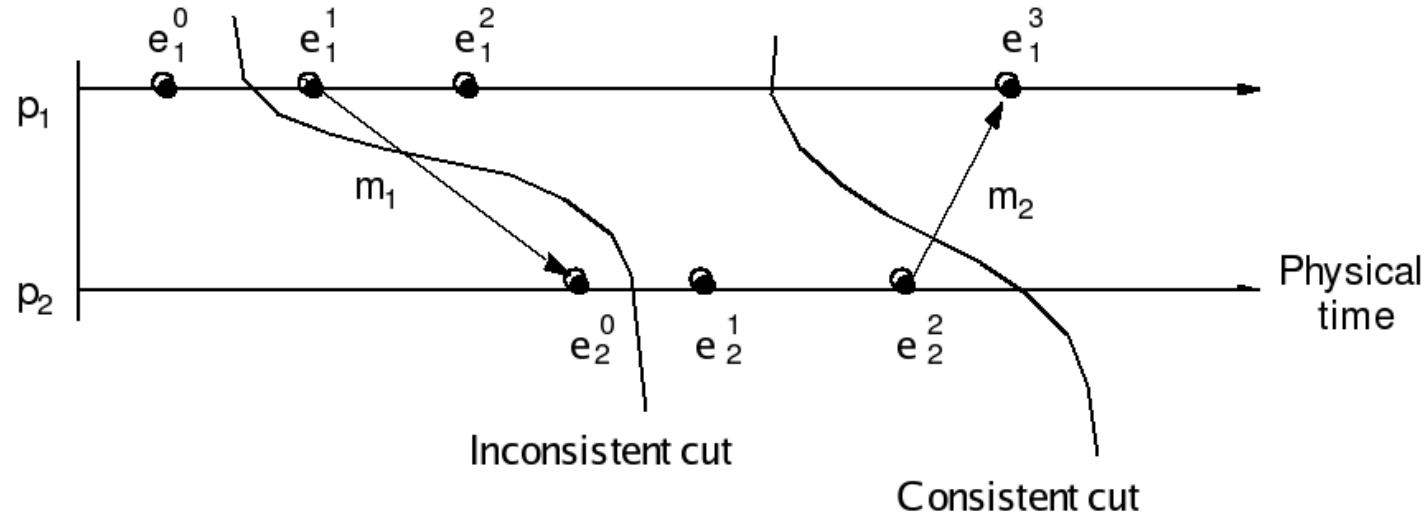
- Weaker form: global state =
combination of local states of processes and channels
at some time and evolution which could have occurred.
- Why do we want global state? Important use cases:
 - finding lost token
 - termination of distributed computation
 - garbage collection.

We assume a strongly connected network.

“Consistent State”

Intuitively: **consistent global state** =
“snapshot” that looks to all processes as if it was
taken at the same instant, everywhere.

- Causal relations must be observed: **consistent cut**



Consistent Cut

Global state = must be a consistent cut

Property:

- Either:
all events before the cut must have happened before the events after the cut,
- Or/except: the events are unrelated (“concurrent”)
- This can be checked with vector clocks.

The big problem are the “messages in flight” (in some buffer): they are part of the state.

Distributed Snapshot

Snapshot algorithms to record a consistent state

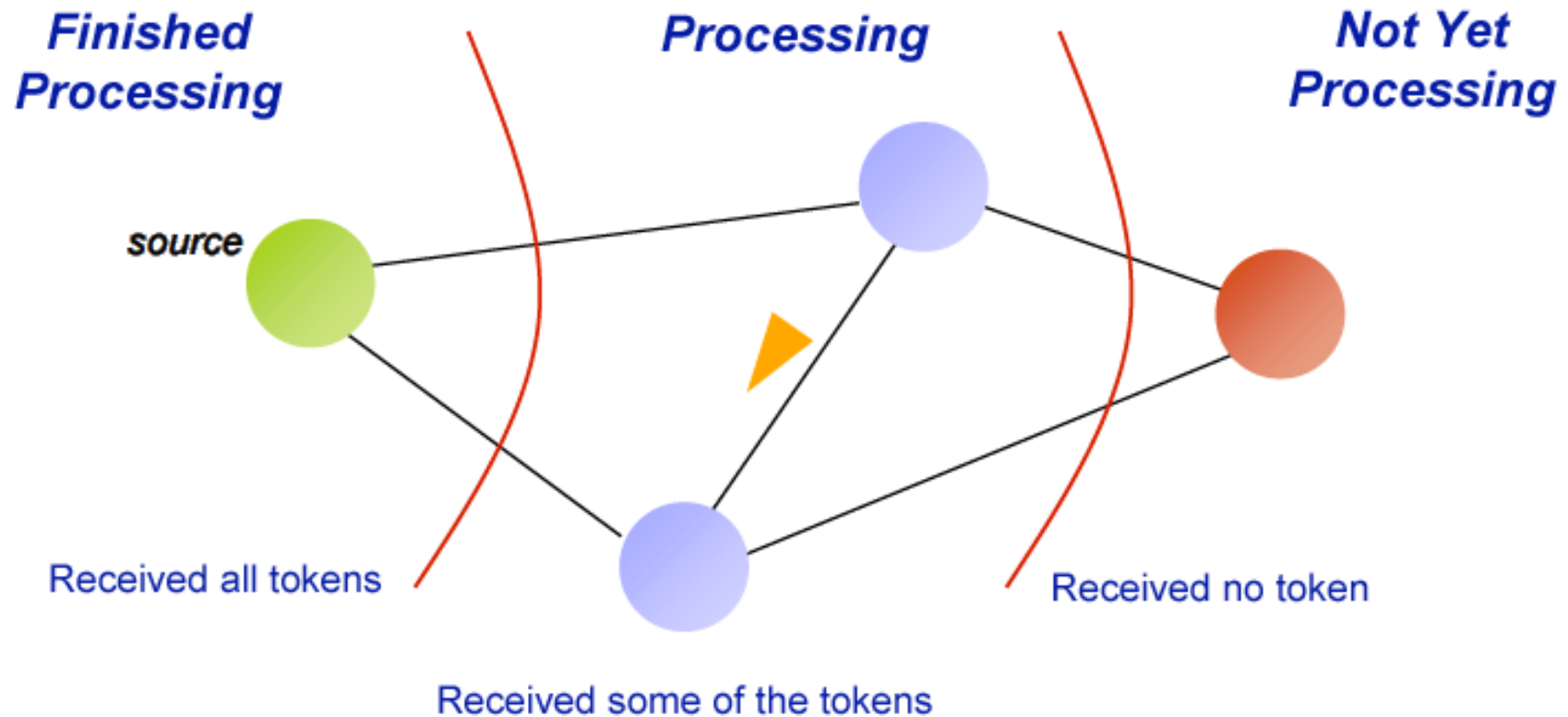
- Algorithm of Chandy and Lamport:
 - creates one (possible) consistent cut
 - in a distributed manner
- Assumptions made by Chandy and Lamport:
 - reliable message exchange, FIFO, unidirectional
 - no failures (of nodes, links)
 - strongly connected graph

Chandy and Lamport (1985)

Idea: “drain” all messages pending in the channels, put a marker before and after these messages.

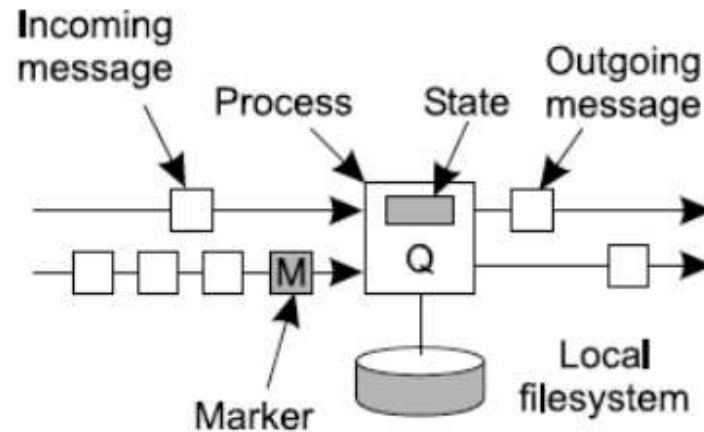
- Initiate snapshot by sending a marker M
- When receiving M
 - stop processing, record own state
 - send out marker on all links
 - record all subsequently received messages (except where M arrived)
 - until a next marker is received on each link.

Chandy and Lamport: Intuition



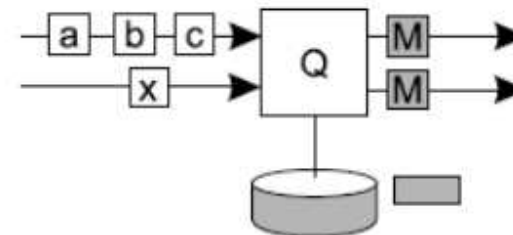
“Token wave” which flushes the channels (two frontiers)

Chandy and Lamport: Example at one Node

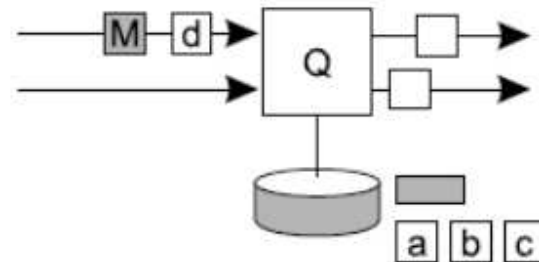


Normal processing, first marker about to be received

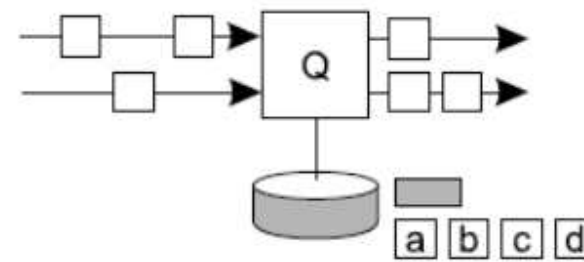
Processing



Token just received for the first time, state saved, token forwarded to outgoing links, begin recording messages



Recording of messages from the incoming links from which a token has been received



Token received, state recording is stopped

Recorded state

Overview

- Distributed snapshot
- **Byzantine agreement**
 - two general's problem
 - byzantine failures
 - byzantine problem, (impossibility) results
- Group communication:
 - Reliable Broadcast

Byzantine Agreement and related topics

Presentation approach:

- Failure modes (including “byzantine failures”)
- Problematic consensus in case of message loss (“two generals’ problem”)
- Problematic consensus in a group, with reliable messages (byzantine agreement problem, $3f + 1$ requirement)
- Impossibility result for asynchronous settings (Fischer, Lynch and Peterson, 1982)
- Discussion: ways out of this problem

Failure Modes

Depending on the assumptions of failure, different algorithms (or impossibility results) will ensue.

- **Fail-stop:** (although bad, this is the ‘nice one’)
 - either a processor works correctly and participates
 - or it has failed and will never respond again.Moreover: the others can reliably detect failed processors.
- **Slowdown:** when some processors execute slowly (or fail), the others cannot know for sure.
- **Byzantine:** every action might be corrupt (see next slides)

Fail-stop is often assumed, but is not realistic (e.g. msg delay)

The Byzantine Failure Mode

Byzantine failures include everything:

- lost messages
- crashing nodes
- faulty implementation
- **malicious implementation, even collusion!**

In the following we make things simple:

- synchronous execution model
- reliable message exchange
- full connectivity and sender identity

That is: “only” the processors behave byzantine, it’s problem enough

Byzantine Settings

Overview of subsequent slides:

- The “Two Generals’ Problem”
- traitors in the network
- $3f + 1$ (= # of nodes needed if we have f traitors)

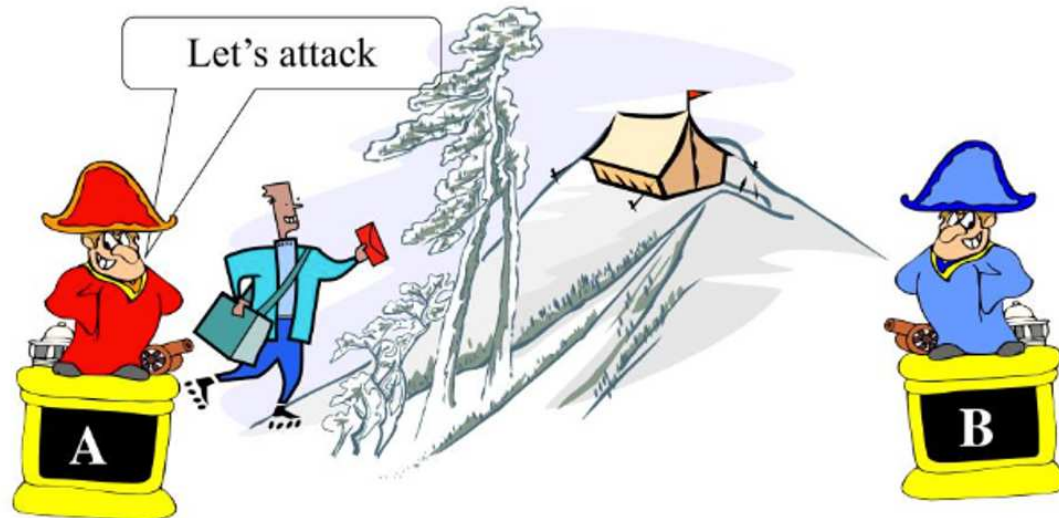
The “Two Generals’ Problem”

First traces of the problem in 1975, mostly cited from Jim Gray’s book in 1978.

What if message passing is not reliable? Big trouble!

(Epistemological)
Statement:

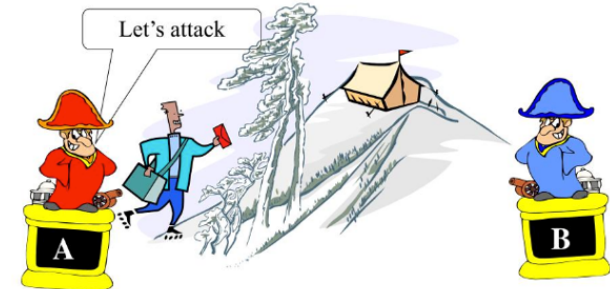
It is impossible
to establish shared state
with certainty.



<https://image1.slideserve.com/1797755/the-2-generals-problem-1.jpg>

Two Generals' Problem, Story

Two generals must coordinate their attacks for if they do not attack together at the same time then they will be defeated for sure.



- Both will attack *only if certain* that the other agreed on the attack time.
- But no message exchange protocol is able to provide ultimate certainty:

→ “I suggest to attack tomorrow at 8am, please confirm”

(otherwise I will not attack)

“It’s confirmed for tomorrow 8am. Please confirm reception” ←

(otherwise I will not attack)

→ “I confirm reception. Please confirm reception of confirmation”

(otherwise I will not attack)

“Confirmed. Please confirm reception of this confirmation 2” ←

(otherwise I will not attack)

→ “I confirm reception. Pls confirm recpt of my confirm. for your confirmation 2”

(otherwise I will not attack)

etc etc

Two Generals' Problem, Insight

Ultimate certainty about having successfully synchronized state **and** knowing that the other side arrived at the same conclusion, is impossible.

- applies e.g. to TCP (ending a session)
- any other state synchronization task with side effects
("was some action executed and does the other side know it, so that I can purge state?")

In practice: must resort to "good-enough solutions"

Byzantine Agreement Problem

The story:

- Romans still rule over Constantinople, but Ottoman battalions want to launch an attack.
- Only a coordinated attack will be successful, a coordinated retreat is also an option.
- The general (there is only one) decides to attack (or retreat): all lieutenants receive the order, lieutenants will exchange messages to verify the outcome.
- Some lieutenants (incl the general) are traitors, and pass on lies.

Is there a protocol for (successfully) coordination?

Also known as: **consensus algorithms, interactive consistency algorithms**

Byzantine Agreement Problem, CS version

- Input: Each process starts with a bit
- Goal: run a protocol such that
 - eventually, all processes output the same bit
 - the output bit must match at least one of the initial bits
- Q: how many faulty processes can you tolerate?

The problem was proposed in 1980
by Lamport, Shostak and Pease.

Byzantine Agreement: Desired Properties

- Non-trivial protocol:
if all lieutenants have same input b , the loyal lieutenants' agreement must be b .
- Core property:
All loyal lieutenants agree on the same value
(which must not be original b , but it must be the same for all)
- Protocol must terminate.

Byzantine Agreement: Impossibility (1)

Theorem: **There is no protocol for byzantine agreement among 3 nodes if at least one node fails.**

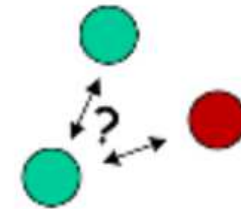
- Applied to story:
with three lieutenants, already one traitor lieutenant will prevent successfull agreement
- Plausibility (see next slide)
- More extensive argument (just afterwards):
 - we build a 6-node network of reliable processes
 - on this covering graph we **emulate** (malicious) failures
 - ie, some processes output (deliberate) wrong answers

Plausibility of the $3*f+1$ Requirement

a) $N = 3, f = 1$

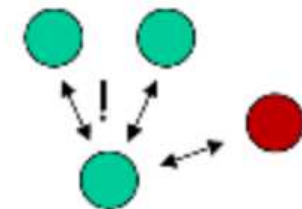
each node is confronted with two peers:

impossible to tell which one is the **traitor**

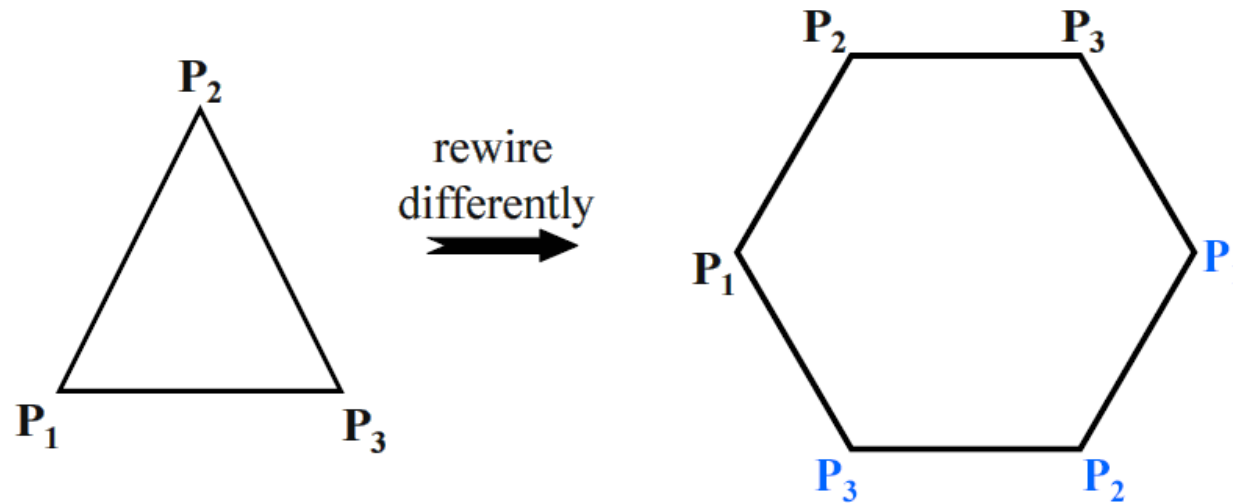


b) $N = 4, f = 1$

each node is confronted with *three* peers: if there is at most one traitor (and we are not this traitor), then the two other loyal nodes will have a majority with their claims.



Byzantine Agreement: Impossibility (a)

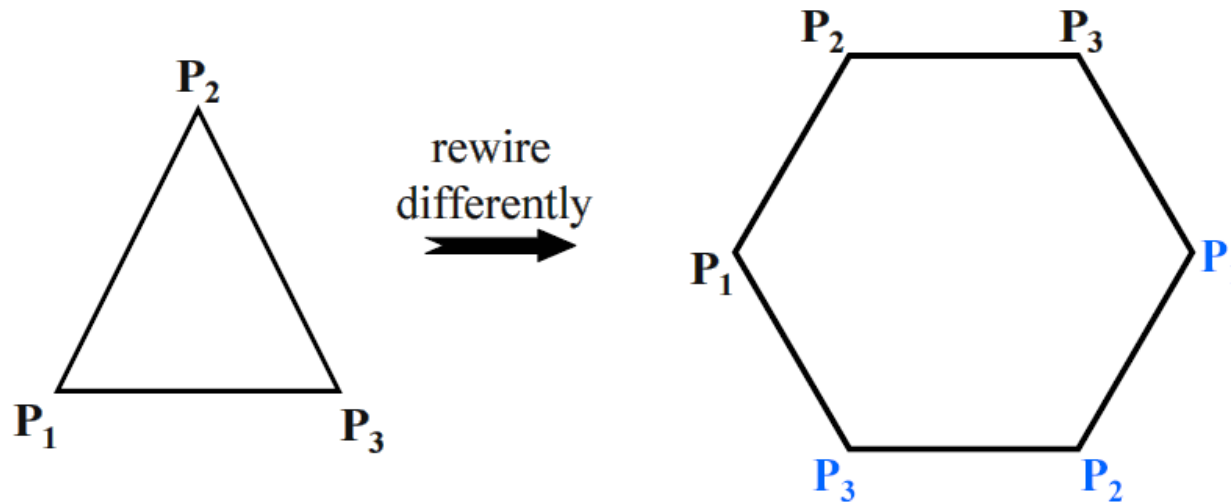


Figs from G. Candea: “Covering graph”

- a **blue process** P_x runs identical code as the black P_x
- broadcast (left) is represented as point-to-point (right)

This will allow us to represent the behavior of malicious nodes

Byzantine Agreement: Impossibility (b)



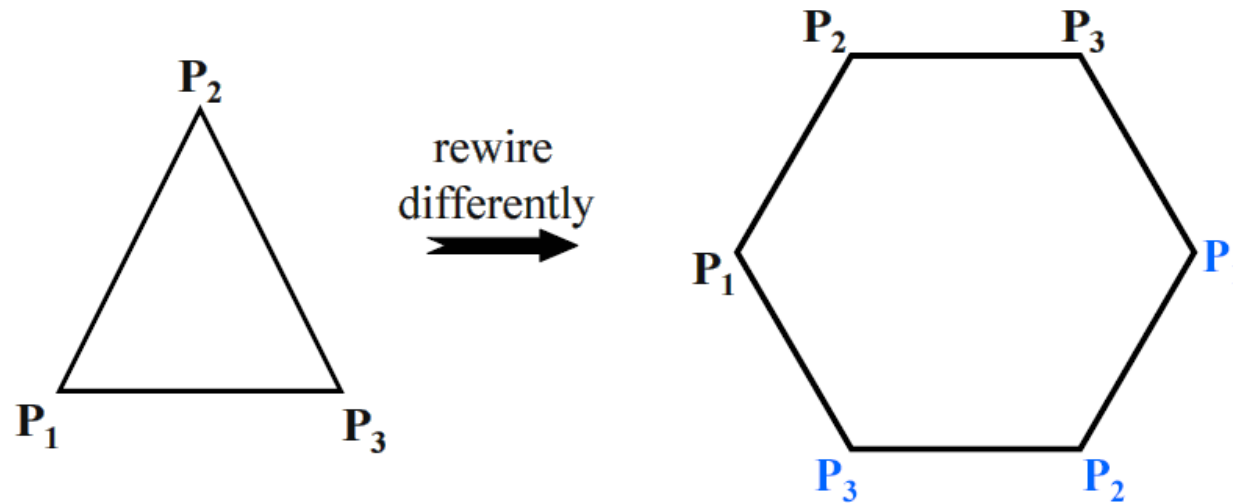
Reading this graph:

- consider any pair of (honest) nodes (e.g. P_2, P_3)
- this pair talks to two versions of third node (P_1)

Being malicious/faulty means:

- the “two P_1 processes” send different bits to P_2 and P_3

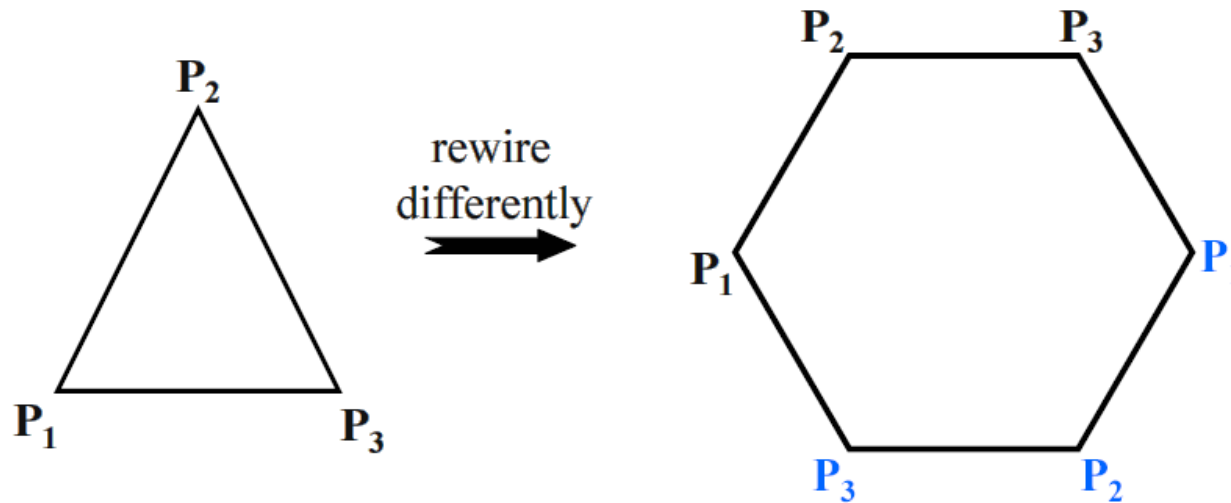
Byzantine Agreement: Impossibility (c)



Recap of the Byzantine Agreement Problem:

- Assume P_2 and P_3 are honest, but talk to malicious P_1 first
- then P_2 and P_3 talk to each other, find conflicting bits
- P_2 cannot infer whether P_3 or P_1 is malicious, same for P_3

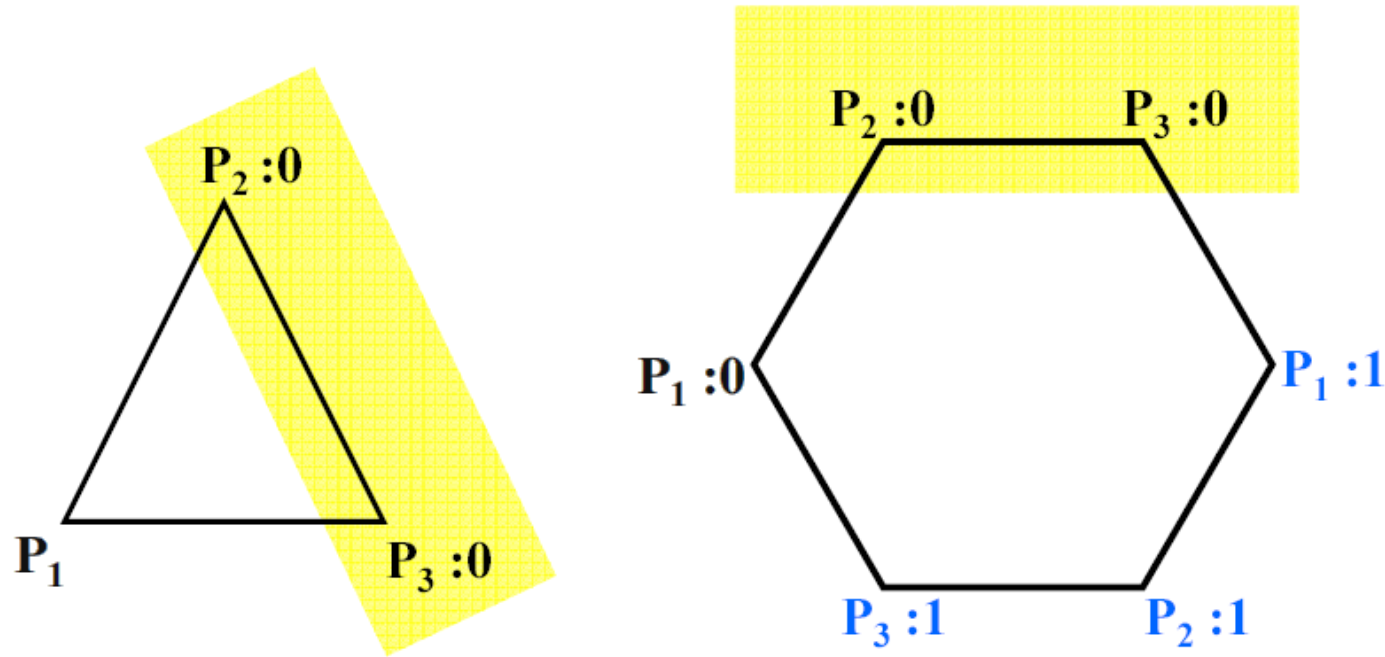
Byzantine Agreement: Impossibility (d)



Assumption:

- $\exists$ byz. agr. protocol for three nodes and one failure
- we will color the honest pair of nodes
- we will use : 0 and : 1 to show the bits known to a process

Byzantine Agreement: Impossibility (case 1)

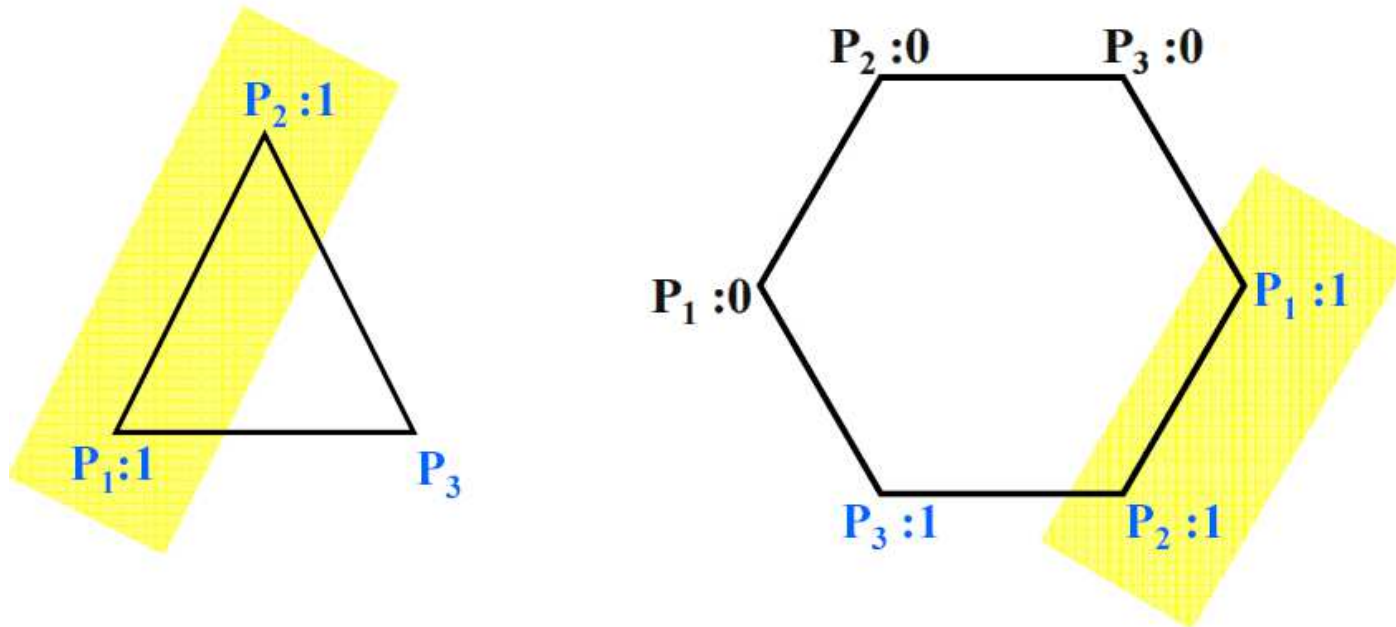


Given: P_2 and P_3 have initial input 0

What happens if P_1 is faulty? $\rightarrow$ P_1 sends different values

P_2 and P_3 nevertheless agree on 0 (assumption!)

Byzantine Agreement: Impossibility (case 2)

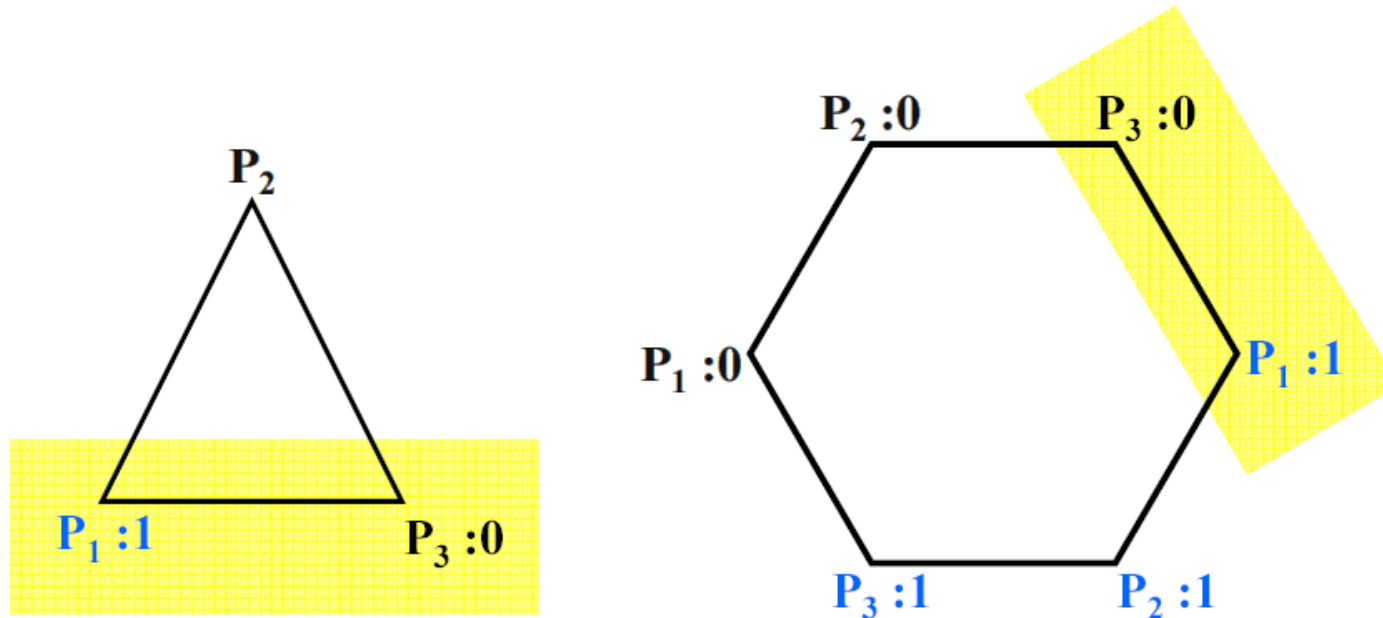


Given: P_1 and P_2 have initial input 1

What happens if P_3 is faulty? $\rightarrow$ P_3 sends different values

P_1 and P_2 nevertheless agree on 1 (assumption!)

Byzantine Agreement: Impossibility (case 3)



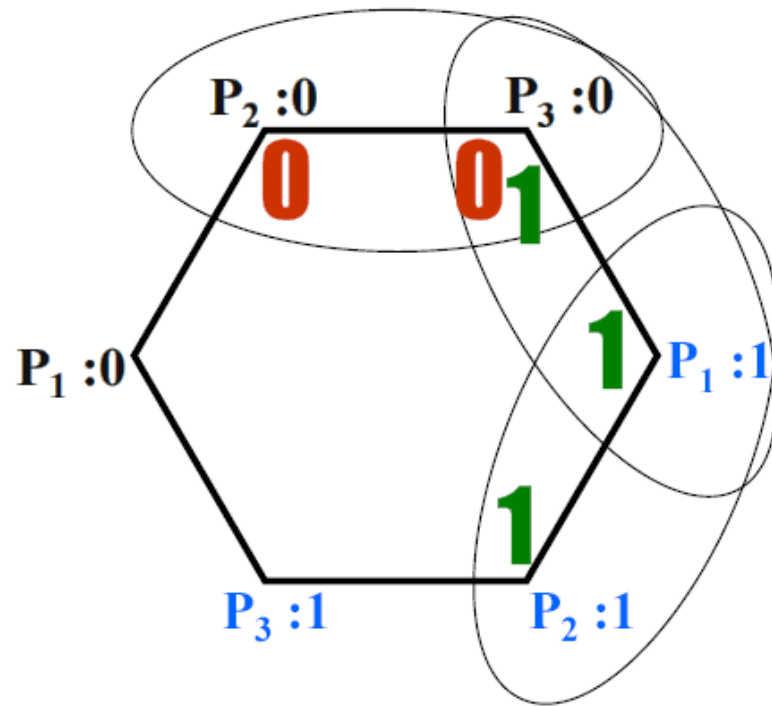
Given: P_1 and P_3 have initial input 1 and 0, respect.

What happens if P_2 is faulty? $\rightarrow$ P_2 sends different values

P_1 and P_3 nevertheless agree on *some* value (assumption!)

Byzantine Agreement: Impossibility (combine cases)

Trying to harmonize the different views results in contradiction:



- **P1** and **P2** agreed on 1
- **P1** and **P3** agreed on some value; because **P1** above went for 1, this must be 1 also for **P3**
- However: **P3** already agreed on 0 (with **P2**)
- contradiction.

Byzantine Agreement, Theory Results

- Theory result:
Protocols exist, if we have $3f+1$ nodes for f failures
- **But:** Result is different in asynchronous model:
No protocols at all! A single faulty node is the end!
The “celebrated result” of Fischer, Lynch and Peterson in 1982)

In 2007, more than 1200 follow-up publications . . .

Protocol variations (ways out of impossibility result)

Approach: change some assumptions, use rules and tricks
randomized instead of deterministic algorithm, or private channels, or signed messages, or failure detectors (instead of proceeding despite failures).

- Ben-Or (1983): exponential (in time) asynchronous byzantine agreement, works for $f < n/5$, still open whether can be improved
- Castro and Liskov (1999): “Practical Byzantine Fault Tolerance”

Today, in practice: ready-to-go implementations. Famous but complex **Paxos** protocol (in the cloud, but also Ethernet switches), other systems like RAFT ... and Bitcoin.

Overview

- Distributed snapshot
- Byzantine agreement
- **Group Communication:**
 - Reliable Broadcast

Group Communication – as middleware

We have datagrams (reliable, unreliable) and remote procedure calls: why yet another type of communication primitives?

- Consensus theory (for example):
want library of useful distributed services
- Working with sockets and RPC (next lecture) is too low-level
- “Middleware”: site between OS and (distributed) application processes and provides highlevel services

Group Communication – issues

Distributed (coordination) services come in many variants:

- Offer different failure semantics and aggregates:
 - send reliably to *all* processes, or discard
 - have all processes receive msgs in *same* order . . .
- Design issues:
closed/open groups, group membership problem,
group addressing, atomicity (all-or-nothing),
msg ordering, scalability.

No “universal” (and final) set of standard groupware services, yet.

Group Communication – “groups”

Example Service: How to agree, inside a defined (distributed) group of processes, on the ordering of events?

- Consensus problem:
In theory, we cannot solve this problem (async!)
- Trick: Modify the group (kick out problematic members), such that the remaining members can be made to agree.
- Messages from kicked-out members are then ignored, as if these nodes had failed.

Group Communication – “tinker” with group membership

Example: Company has drug X against common cold, which **can** cure people within 24h, but it does not always work.

How to turn it into a fully reliable therapy? (great for marketing!)

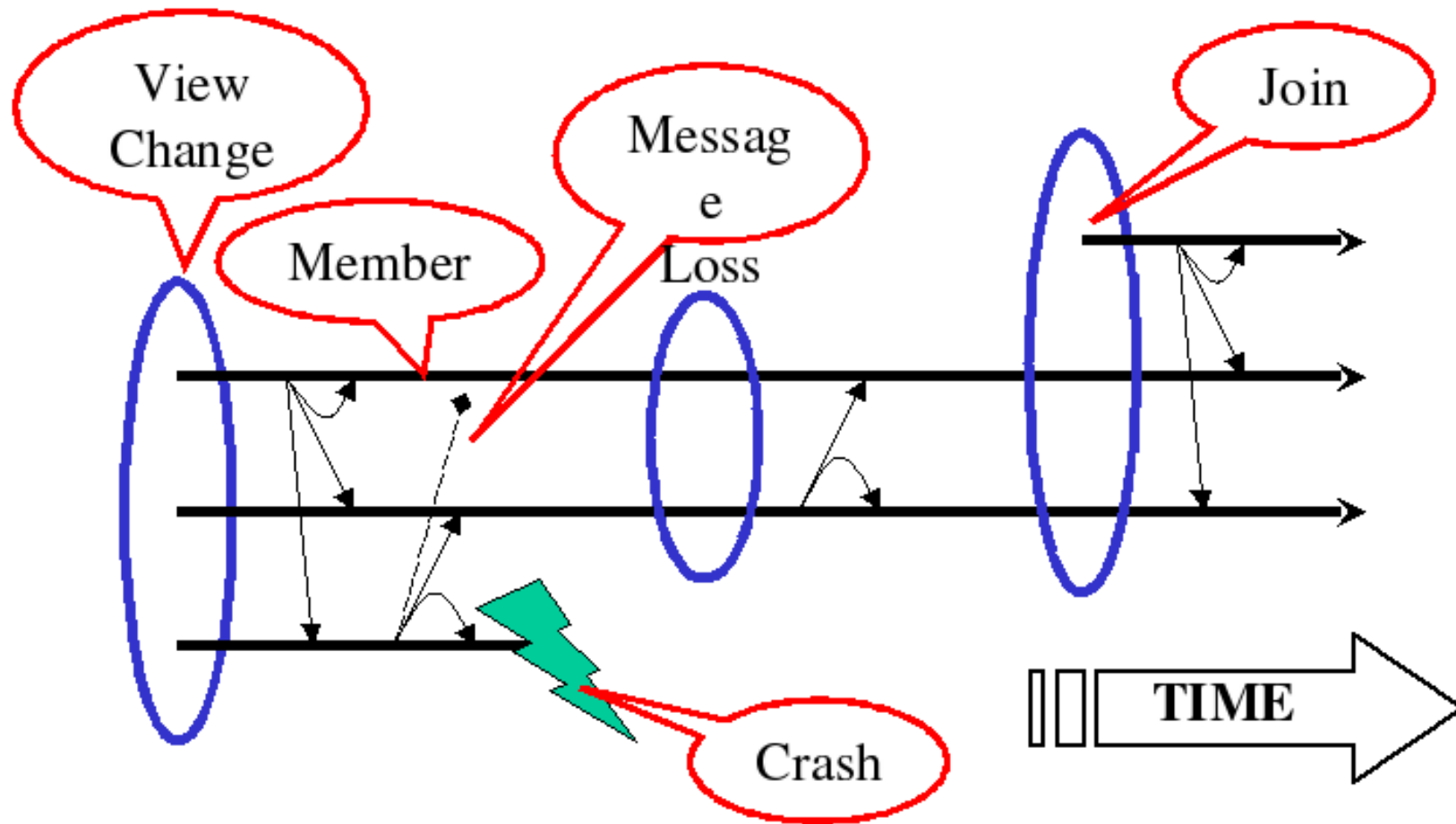
- Add text to the small print: We give guarantee that X works within 24h, *should the patient still be alive at that time.*
- Implementation: Company observes all patients
 - examines situation at 23h 59'
- If cure did not work, let a hit man “solve” the problem.

Forming Groups

What is a group?

- Finite set of peers, explicit membership:
 - nodes have to join
- Primitives:
 - `joinGroup("group", event-handler)`
 - `sendPoint2Point("group", member-id, message)`
 - `multicast("group", message)`
 - `leaveGroup("group")`
- Events can be:
 - `recv`, `join`, `leave`, `crashed`, `view changed`

Group Communication: Graphical Representation



(from R. van Renesse)

Overview

- Introduction Group Communication:
 - **Reliable Broadcast**

Group Communication: Reliable Broadcast

Problem: Implement “all-or-nothing” property for reliable message delivery inside a group.

- “All-or-nothing” property: once a message has been delivered to one destination, it must be delivered to all.
- Useful in databases:
 - send updates to replica servers
 - distributed games: agree how virtual world looks
- Note:
 - all-or-nothing implies consensus
 - **or** reduction of member set

Group Communication: Reliable Broadcast (2)

Protocol (with $O(n^2)$):

- sender S multicasts msg M to all other members
- for every new msg M , each members multicasts it again
 - Note: if there are no message losses,
all nodes receive a copy of M from all other nodes
 - Assume some node A does not get M , at all:
 - A will not be able to re-broadcast
 - the others will notice, and will send it a copy
 - If, after some time, A still did not re-send M ,
the others agree to kick out A .

Group Communication: Reliable Broadcast (3)

Another protocol, similar to two-phase-commit:

- Sender S multicasts “prepare to commit M” message
- all node store M, send back an acknowledgement
- if all acks arrive, S confirms transaction
- if some ack is missing, S starts again
- if a process never sends an ack, it is declared dead before restarting the transaction.

Note that it's a consensus problem, with **failure detector**!

Other Group Communication Primitives

Seen so far: reliable broadcast. Many other group comm services:

- FIFO broadcast – msgs from same sender are delivered in order
- Causal broadcast – only causal relationship is preserved
- Barrier – wait until all reached the barrier
- Scatter-gather – send val, all compute, all receive all results.
- Termination detection
- and more . . .